DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

PgBouncer's second startup message, and Databricks makes BEGIN mean something on the lakehouse

Issue #14 | Sun, 18 Oct 2026 | ⏱️ ~33 min read | Level 3/5

🗄️ Today's two stories

1. **PgBouncer 1.26.1 fixes an authentication bypass that lets anyone with one SCRAM user's password log in as any other SCRAM user, admins and superusers included.** It's the third security release in five months, three weeks after the one we covered in Issue #3. We explain the PostgreSQL startup and SCRAM handshakes, how a second startup message breaks the link between "who is proving" and "who is logging in", which versions and configs are affected, and what to do if your distro or cloud hasn't shipped the fix.
2. **Databricks says multi-statement, multi-table transactions are generally available on Unity Catalog managed tables.** `BEGIN ATOMIC ... END` and `BEGIN TRANSACTION ... COMMIT` now work across up to 100 tables. We explain why this was hard on files in object storage, how catalog commits move the "who wins" decision from the file system into the catalog, how conflicts are detected, and the limits that matter, including a docs-versus-blog disagreement about Iceberg.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 PgBouncer's second startup message

PGBOUNCER 1.26.1 · RELEASED 14 OCT 2026 · 3 CVES, ONE AN 8.8 AUTH BYPASS · ~14 MIN READ

Three weeks ago we told you to upgrade PgBouncer to 1.26.0 because two of its three new CVEs could be triggered by anyone who could reach the port. If you did, thank you. Now do it again. PgBouncer 1.26.1 fixes a bug that's worse than anything in the last release: a client that knows the password of one ordinary user can end up logged in as a different user, including your PgBouncer admins and your Postgres superusers. The trick is almost absurdly simple. Start logging in as one person, then, halfway through, send the "hello, I am…" message again with someone else's name.

WHAT HAPPENED

On **14 October 2026**, the PgBouncer project released **version 1.26.1**, which "fixes three CVEs".[1,2] The release notes describe them like this:[1,3]

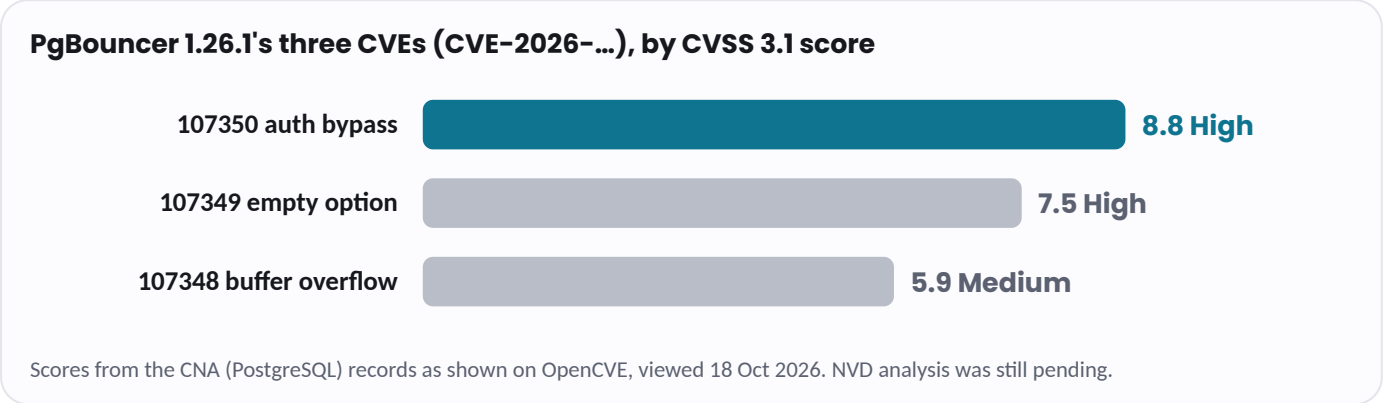
- **CVE-2026-107350, authentication bypass.** A client "that knows the password of one SCRAM user" can "log in as any other SCRAM user, including admins and backend superusers". The cause is "a second StartupMessage being processed during SCRAM authentication when connecting to a non-existent database". Affected: **1.17.0 through 1.26.0**.

- **CVE-2026-107349, denial of service.** An unauthenticated client can put PgBouncer into an infinite loop with an empty option name in the startup `options` parameter, when `ignore_startup_parameters` is set but doesn't include `options`. Affected: **1.20.0 through 1.26.0**.
- **CVE-2026-107348, denial of service.** An integer overflow in the packet buffer growth logic causes an infinite loop, but only when `max_packet_size` is set above its default. Affected: **1.21.0 through 1.26.0**.

The CVE records, published the same day with the PostgreSQL project as the **CNA** ≈ *CVE Numbering Authority: the organization allowed to assign CVE IDs and publish the official record for a product*, rate the bypass **8.8 (High)** with the vector `AV:N/AC:L/PR:L/UI:N`: reachable over the network, low complexity, needs a low-privileged login, no user interaction.[4,5] The empty-option loop scores **7.5** and needs no login at all; the overflow scores **5.9** because it needs a non-default setting.[6,7]

The credits are worth reading. The bypass was found by **Dave Aitel of OpenAI**, the empty-option loop by **Jackds Security**, and the overflow by **Brian Cosgrove**.[5,6,7] All three IDs were reserved on 7 October, a week before publication.[5,6]

The release also fixes a regression from 1.26.0. That version started tracking `search_path` by default, which broke the common setting `ignore_startup_parameters = search_path`. In 1.26.1, `ignore_startup_parameters` "take[s] precedence over tracked parameters", restoring the 1.25 behaviour.[1]



THE BACKGROUND YOU NEED

What PgBouncer does, and why it holds everyone's keys

A Postgres server starts a separate operating-system process for every connection, so thousands of idle client connections are expensive. A **connection pooler** ≈ *a proxy that accepts many client connections and shares a smaller number of real database connections among them* sits in between. PgBouncer is the most widely deployed one. Clients connect to it as if it were Postgres, and it hands them a server connection from a pool. In the default `session` **pool mode** ≈ *PgBouncer's rule for when a server connection goes back to the pool: after the client disconnects (session), after each transaction (transaction) or after each statement (statement)*, the server connection goes back when the client disconnects; in `transaction` mode, at the end of every transaction.[8]

Because clients never talk to Postgres directly, **PgBouncer has to authenticate them itself**, and then log in to Postgres on their behalf. As CYBERTEC's Laurenz Albe puts it, PgBouncer's own HBA file "only determines how

clients have to authenticate to pgBouncer"; the server-side login is a separate step.[9] PgBouncer gets the password material it needs in two ways:[8,9]

- an **auth_file** ≈ *a text file (often userlist.txt) listing user names and their password hashes or SCRAM secrets, which PgBouncer loads at startup*, and/or
- an **auth_query** ≈ *a SQL query PgBouncer runs against Postgres, as a dedicated auth_user, to fetch a connecting user's password hash on demand*. The default reads `rolpassword` from `pg_authid`.

PgBouncer's docs explain why the SCRAM secrets in that file are so sensitive. They're reused "as the passwords for outgoing connections to the backend server", which works only "if the client authentication also uses SCRAM" and "the SCRAM secrets are identical in PgBouncer and the PostgreSQL server".[8] So a PgBouncer instance is a single process that knows how to log in to your database as every user it serves. It also has an **admin console** ≈ *a virtual database named pgbouncer where users listed in admin_users can run commands such as SHOW POOLS, PAUSE and SHUTDOWN*, open to whoever is listed in `admin_users`.[8]

One more detail matters today: PgBouncer is a single process. Christophe Pettus's post on 1.26.0 noted that a hang "stops all clients", because every connection shares that one process.[10]

How a Postgres login starts

Every Postgres connection begins with the client sending a **StartupMessage** ≈ *the first message of the Postgres wire protocol, carrying the protocol version, the user name, the database name and optional run-time settings*. The protocol docs say it "includes the names of the user and of the database the user wants to connect to", plus optional run-time parameters.[12] One of those parameters, `options`, carries command-line style settings such as `-c search_path=app`. PgBouncer only lets through parameters it can track, and `ignore_startup_parameters` lists others it should silently drop.[8]

The server then picks an authentication method. For SCRAM, the exchange is a fixed sequence:[12,13]

1. Server sends **AuthenticationSASL**, listing mechanisms.
2. Client sends **SASLInitialResponse** containing the SCRAM *client-first-message*.
3. Server sends **AuthenticationSASLContinue** with the *server-first-message*.
4. Client sends **SASLResponse** with the *client-final-message*, which contains the proof.
5. Server sends **AuthenticationSASLFinal** and then **AuthenticationOk**, or an error.

Note what's missing: a user name in the SCRAM part. The Postgres docs say the server "will ignore the user name that the client sends in the client-first-message" and uses the one from the startup message instead.[13] **Who you are is decided once, at the start. Everything after that proves it.** Remember that sentence.

What SCRAM actually proves

SCRAM-SHA-256 ≈ *Salted Challenge Response Authentication Mechanism: a login protocol in which the client proves it knows the password without sending it, and the server never stores the password itself* has been Postgres's default password

hashing (`password_encryption`) since version 14.[10] The server stores, for each user, a salt, an iteration count, a **StoredKey** and a **ServerKey**. RFC 5802 defines them:[14]

```
SaltedPassword  := Hi(Normalize(password), salt, i)
ClientKey       := HMAC(SaltedPassword, "Client Key")
StoredKey       := H(ClientKey)
ClientSignature := HMAC(StoredKey, AuthMessage)
ClientProof     := ClientKey XOR ClientSignature
```

The `AuthMessage` is the conversation so far, including the random nonces from both sides. To check a login, the server computes `ClientSignature` from its StoredKey, XORs it with the client's proof to recover a candidate ClientKey, hashes that, and compares it to the StoredKey.[14] A match means the client knows that user's password.

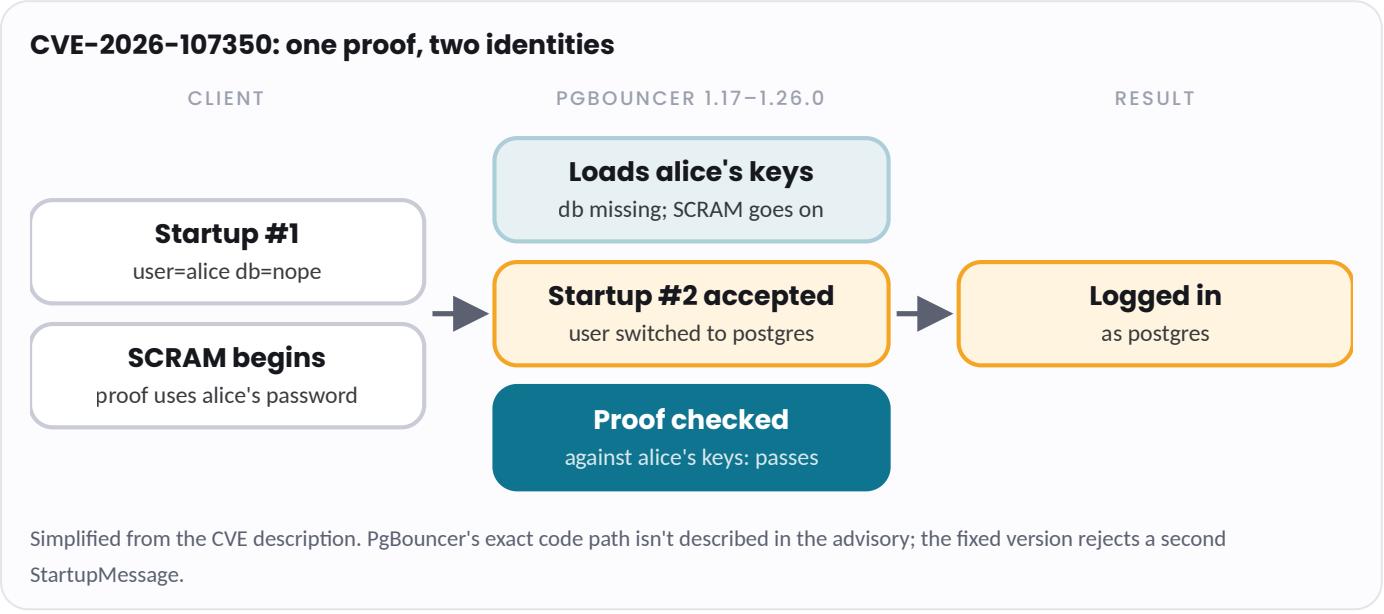
The weak point is the word *that*. The proof is checked against **whichever StoredKey the server looked up**. If the server looks up Alice's keys, and Alice's password is right, the proof passes. The math has no idea whose name is on the session.

HOW IT WORKS

The bypass, step by step

The CVE record describes the flaw in one sentence: a second StartupMessage "sent during SCRAM authentication against a non-existent database is not rejected and switches the user being logged in as, while the SCRAM exchange is still verified against the keys of the first user."[4]

Put the pieces together and the attack writes itself. The attacker knows the password of an ordinary user, call her `alice`, whose SCRAM secret PgBouncer has. They want to be `postgres`.



Simplified from the CVE description. PgBouncer's exact code path isn't described in the advisory; the fixed version rejects a second StartupMessage.

1. The attacker connects and sends a StartupMessage as `alice`, naming a database that doesn't exist.
2. PgBouncer looks up alice's SCRAM keys and starts the exchange.

3. Mid-exchange, the attacker sends a **second** StartupMessage with `user=postgres`. A Postgres server would never expect one; PgBouncer, in this state, processes it and switches the user.
4. The attacker finishes the SCRAM exchange using alice's real password. PgBouncer verifies the proof against the keys it loaded in step 2, alice's, so it passes.
5. PgBouncer now considers the client authenticated as `postgres`.

That's the "split brain" between the two halves of the login. The identity came from the second message; the proof was checked against the first. The release name says it all: 1.26.1 is called **"Only startup once"**.[3]

From there the attacker can, according to the advisory, reach "PgBouncer admin users and backend superusers".[4] An admin console session lets them pause, kill or reconfigure pools. A superuser session in Postgres is game over: it can read any table and change any role.

Who is actually exposed

The CVE says the attacker needs the password of "a user listed in auth_file".[4] OpenCVE's summary of affected configurations is broader: SCRAM-based client auth (`auth_type = scram-sha-256`, `md5` with SCRAM secrets, or matching HBA rules), "with the target user loaded from `auth_file` or `auth_query`".[4] The project's own wording is "any other SCRAM user".[1] Either way, you're in scope if:

- you run PgBouncer **1.17.0 (March 2022) through 1.26.0**,[3,15]
- clients authenticate to PgBouncer with SCRAM, which is likely today given that Postgres has defaulted to SCRAM hashes since version 14,[10] and
- an attacker has, or can guess, **any** user's password. In a shared platform, a SaaS with per-tenant database roles, or a company where every service has its own login, that bar is low.

Notice the "non-existent database" condition costs the attacker nothing: they choose the database name.

The two denial-of-service bugs

The other two CVEs are hangs, and in a single-process pooler a hang is an outage for every client at once.[10]

CVE-2026-107349 is reached before any password check. A StartupMessage whose `options` contain an empty name, the record gives `-c=` as the example, sends PgBouncer into "an endless loop in the ignore_startup_parameters lookup", pinning a CPU core "until it is restarted".[6] It only triggers when `ignore_startup_parameters` is non-empty but doesn't include `options`.[3] That covers any list that doesn't mention `options`, such as `extra_float_digits` alone, or the `search_path` setting the 1.26.1 regression fix is about.

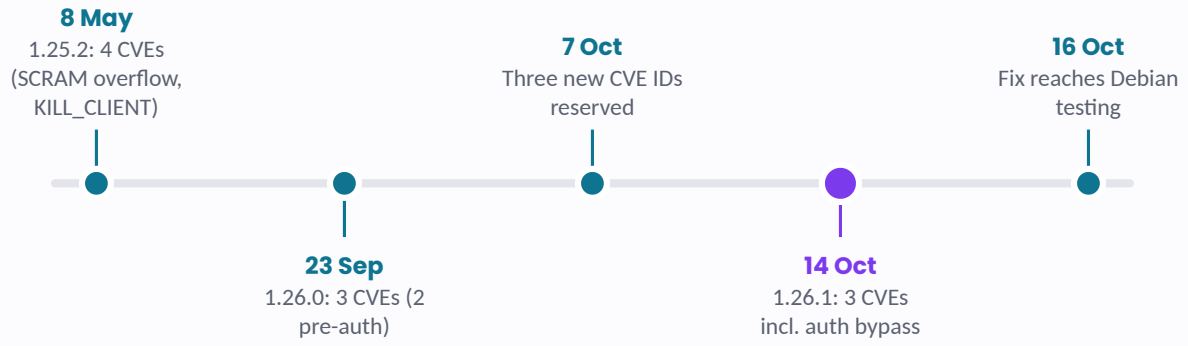
CVE-2026-107348 is an **integer overflow** *≈ when a calculation produces a number too big for its type and wraps around to a small or negative value, which can break size checks* in `mbuf_make_room`, the buffer growth function. A packet header that declares a length above 2^31 leads to a loop that never ends, and since 1.23.0 this is reachable "before authentication using a five-byte packet header".[7] It needs `max_packet_size` raised above the default of 2147483647, which few people do.[8,7] (The sources disagree on the exact threshold, 2147483648 versus

2147483649; either way, the default is safe.[7,3]) It's a cousin of CVE-2026-6668, the buffer-growth overflow fixed in 1.26.0, which *was* reachable at the default.[3]

Five months, ten CVEs

This is now a pattern, and it's worth seeing on one page.

PgBouncer security releases, 2026



Sources: PgBouncer changelog and GitHub releases, CVE records (OpenCVE, ThreatInt), Debian package tracker.

Version 1.25.2 in May fixed four CVEs, including a pre-authentication crash from "an integer overflow in the SCRAM authentication packet parser" and a missing authorization check on the `KILL_CLIENT` admin command.[15,11] Version 1.26.0 in September fixed three more, including a crash from a missing nonce check in the SCRAM client-final-message that had been present since 1.11.0 and was reported by six people independently.[3,10] And now three again.

In May, Pettus wrote that this was "the second SCRAM-handling bug class in PgBouncer within a year", suspected that the state machine around `client_login()` had "grown too complex", and expected a third.[11] This week's bug is, literally, a state machine accepting a message in a state where it shouldn't.

Why so many, so fast? Part of the answer is the one we covered in Issue #13: AI-assisted bug hunting has made old protocol code much cheaper to audit, and the PostgreSQL project itself has been through a record wave of security fixes this year. The credit on today's worst bug goes to an OpenAI researcher.[5] The point isn't that PgBouncer is badly written. It's that a small, critical C program that parses untrusted network input is exactly what these tools are good at, and every pooler, proxy and driver is getting the same scrutiny.

WHAT IT MEANS FOR YOU

Upgrade to 1.26.1. It's a patch release on the 1.26 line; the only behaviour change is the `ignore_startup_parameters` precedence fix, which, if anything, undoes a 1.26.0 surprise.[1] If you're still on 1.25.x or earlier, read our Issue #3 notes on 1.26.0 first: it removed online restart (`-R`), `SHOW FDS` and `SUSPEND` , and it stopped `search_path` and `default_transaction_read_only` leaking between clients in transaction pooling.[3,10]

Check what your packages actually ship. As of 18 October, Debian unstable and testing have 1.26.1, but stable (trixie) carries **1.24.1-1+deb13u2** and oldstable (bookworm) **1.18.0-1+deb12u1**, both with all three

CVEs listed as open.[16] The PostgreSQL community's own apt and yum repositories are the usual faster route. If you use Azure Database for PostgreSQL's built-in PgBouncer, its docs (last updated 22 July) list **1.25.2**; Azure says it manages updates but doesn't publish a schedule.[17] Container images are the classic blind spot: `docker exec <container> pgbouncer --version` is the quickest check you'll do this week.

If you can't upgrade today, you can shrink the exposure. The config below shows the idea.

- **For the bypass:** restrict who can reach the port (OpenCVE relays Red Hat's suggestion to firewall 6432/tcp or bind to 127.0.0.1), and keep privileged roles out of `auth_file`.[4] If the superuser isn't a user PgBouncer can log in as, the worst case shrinks. It doesn't go away: an attacker can still become any *other* user it knows.
- **For the empty-option loop:** either clear `ignore_startup_parameters` or add `options` to it; the record lists both as workarounds.[6] Adding `options` means PgBouncer will drop clients' `-c` settings, so check nothing depends on them.
- **For the overflow:** leave `max_packet_size` at or below the default.[7]

Stopgaps until 1.26.1 is installed (pgbouncer.ini)

Typical config, exposed

```
listen_addr = *
auth_type = scram-sha-256
auth_file = userlist.txt
; every role, incl. postgres:
;   "postgres" "SCRAM-SHA-256$..."
admin_users = postgres
ignore_startup_parameters =
  extra_float_digits
```

Reduced exposure

```
listen_addr = 10.0.1.5
auth_type = scram-sha-256
auth_file = userlist.txt
; superusers removed from
; auth_file and admin_users
admin_users = pgb_admin
ignore_startup_parameters =
  extra_float_digits, options
```

Mitigations from the CVE records and OpenCVE/Red Hat guidance. They narrow the attack; only the upgrade fixes it.

💥 Watch out for

- **The fixed version is 1.26.1.** OpenCVE's auto-generated remediation text says "upgrade to 1.27.0 or later", which doesn't exist. The CNA's version ranges end before 1.26.1, and the vendor release fixes all three.[4,6]
- **"Needs a password" isn't comforting.** Any low-privilege login works: a reporting user, a leaked CI credential, a tenant's own role.
- **Don't confuse this with Postgres.** PostgreSQL itself, which also ignores the SCRAM user name, isn't affected. The bug is in PgBouncer's handling of a second startup message.[13,4]
- **Managed "PgBouncer-compatible" poolers aren't automatically safe or unsafe.** If your provider runs actual PgBouncer, ask for the version. If it runs a different pooler, these CVEs don't apply, but the same class of bug might.
- **Debian stable is still unpatched** at the time of writing.[16]

WHAT HAPPENS NEXT

The obvious things to watch are distribution and cloud updates: Debian stable and Ubuntu security pockets, and Azure's built-in PgBouncer, which hadn't documented 1.26.0 yet. Less obvious: PgBouncer's maintainers have now shipped three security releases in five months, all touching the login state machine. Expect more reports. If you run PgBouncer in production, treat it the way you treat Postgres minor releases, with an owner and a schedule. Pettus's line from two weeks ago still stands: PgBouncer "has no such process", and that neglect is the real vulnerability.[10]

📖 Words from this story

CNA: CVE Numbering Authority, the organization allowed to assign CVE IDs and publish the official record for a product.

Connection pooler: a proxy that accepts many client connections and shares a smaller number of real database connections among them.

Pool mode: PgBouncer's rule for when a server connection goes back to the pool: after the client disconnects (session), after each transaction (transaction) or after each statement (statement).

auth_file: a text file (often userlist.txt) listing user names and their password hashes or SCRAM secrets, which PgBouncer loads at startup.

auth_query: a SQL query PgBouncer runs against Postgres, as a dedicated auth_user, to fetch a connecting user's password hash on demand.

Admin console: a virtual database named `pgbouncer` where users listed in `admin_users` can run commands such as SHOW POOLS, PAUSE and SHUTDOWN.

StartupMessage: the first message of the Postgres wire protocol, carrying the protocol version, the user name, the database name and optional run-time settings.

SCRAM-SHA-256: Salted Challenge Response Authentication Mechanism, a login protocol in which the client proves it knows the password without sending it, and the server never stores the password itself.

Integer overflow: when a calculation produces a number too big for its type and wraps around to a small or negative value, which can break size checks.

STORY 2 Databricks makes BEGIN mean something on the lakehouse

DATABRICKS · MULTI-STATEMENT TRANSACTIONS GA, BLOG 16 OCT 2026 · UNITY CATALOG MANAGED TABLES · ~15 MIN READ

Here's a job you've probably written. Step one merges new orders into `orders`. Step two decrements `inventory`. Step three writes a row to `audit_log`. On a database, you wrap those in `BEGIN` and `COMMIT` and stop worrying. On a lakehouse, until recently, each step committed on its own, so when step two failed at 3 a.m. you woke up to orders without stock movements and a pile of "repair" logic to write. Databricks now says that's over: transactions that span statements and tables are generally available. Making that work on top of files in object storage is a neat piece of engineering, and the limits tell you exactly where the seams are.

WHAT HAPPENED

On **16 October 2026**, Databricks announced that "Multi-Statement Transactions are Generally Available for UC managed Delta Lake and Apache Iceberg tables on Databricks."[18] The post, by Benjamin Mathew, Prakhar Jain and Sumeet Varma, claims Databricks is "the first lakehouse architecture to support multi-statement transactions natively across both Delta Lake and Apache Iceberg tables".[18]

The feature went into **Public Preview for managed Delta tables on 4 March 2026**, with managed Iceberg tables in Private Preview.[19] The GA announcement adds that transactions compose with temporary tables, stored procedures and SQL scripting, can write to streaming tables, and respect Unity Catalog ABAC ≈ *attribute-based access control: permissions driven by tags on data and users, used here for row filters and column masks* policies on both reads and writes.[18]

There are two syntaxes. A non-interactive block commits or rolls back as a unit:[21]

```
BEGIN ATOMIC
  MERGE INTO orders USING staged_orders ON ...;
  UPDATE inventory SET qty = qty - ... WHERE ...;
  INSERT INTO audit_log VALUES (...);
END;
```

And an interactive transaction, for when you want to look before you commit:

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- or ROLLBACK;
```

Two customers are quoted. **Fivetran** says its History Mode, which needs two writes per change, now commits "the close and the insert as one atomic unit", removing "an entire class of consistency-repair code". **Accolade** says it processes "roughly 1,700 transactions in production every day" for claims and liabilities records.[18] The

only performance figure is an image caption, "4.7x faster performance of concurrent transactions at scale", with no workload, baseline or method published.[18] Treat that as a marketing claim.

THE BACKGROUND YOU NEED

Why the lakehouse didn't have this

A traditional warehouse has always had transactions. Snowflake, for example, supports `BEGIN` / `COMMIT` / `ROLLBACK` with READ COMMITTED isolation.[27] A **lakehouse** *≈ an architecture that stores tables as open-format files (such as Parquet) in cheap object storage, with a metadata layer that adds database features on top* is different: there's no database server that owns the data. A table is a directory of **Parquet** *≈ a column-oriented file format for analytics; each file stores data column by column, compressed* files plus metadata, and many engines might read and write it.

Delta Lake *≈ an open table format that turns a directory of Parquet files into a transactional table by keeping an ordered log of changes next to them* solves single-table transactions with a **transaction log** *≈ Delta's _delta_log directory: numbered JSON files, one per commit, each listing files added and removed; the table's state is the sum of the log*. Databricks' classic explainer describes it as "an ordered record of every transaction that has ever been performed", with each commit "written out as a JSON file, starting with 000000.json".[24] The rule is: "if it's not recorded in the transaction log, it never happened."[24]

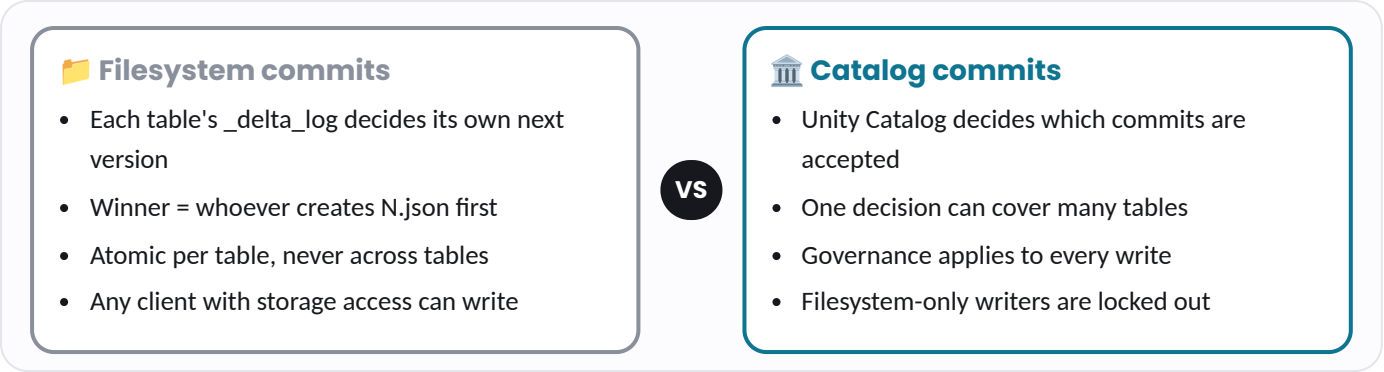
Concurrency works by **optimistic concurrency control** *≈ a strategy where transactions run without locks and check for conflicts only when they try to commit; the loser retries or fails*. A writer records the version it started from, does its work, and tries to create the next log file. Mutual exclusion means "only one user can successfully make commit 000001.json". The loser checks whether anything it read has changed, and either retries on top of the winner or throws an error.[24]

That design has a built-in limit. **The file system decides who wins, one table at a time.** Delta "relies entirely on the filesystem for read-time discovery and write-time commit atomicity", and each table "manages its own transaction log and conflict detection independently".[25] There's no single place that can say "these three commits, to three different tables, happen together or not at all". Apache Iceberg has a similar gap: its REST catalog spec includes an *optional* multi-table commit endpoint, but as Dremio's Alex Merced noted last month, a catalog "may not expose it", clients don't automatically use it, and "catalog atomicity is not the same as distributed transaction isolation".[26]

HOW IT WORKS

Catalog commits: moving the referee

The enabling feature is what Databricks calls **catalog commits** (open-source Delta calls them **catalog-managed tables** *≈ Delta tables whose commits are coordinated by a catalog service rather than by atomic file creation in storage*). Unity Catalog becomes "the single source of truth" for table state and coordinates commits, which lets it "coordinate commits across multiple tables in one transaction".[22]



The open protocol side landed in **Delta Lake 4.0.1** as a `catalogManaged` reader-writer table feature.[25] It needs reader version 3 and writer version 7, plus the In-Commit Timestamps and VACUUM Protocol Check features, and the Delta docs are blunt that "Filesystem-based access to catalog-managed tables is not supported."[25] That last sentence is the price: once the catalog is the referee, nobody gets to write around it.

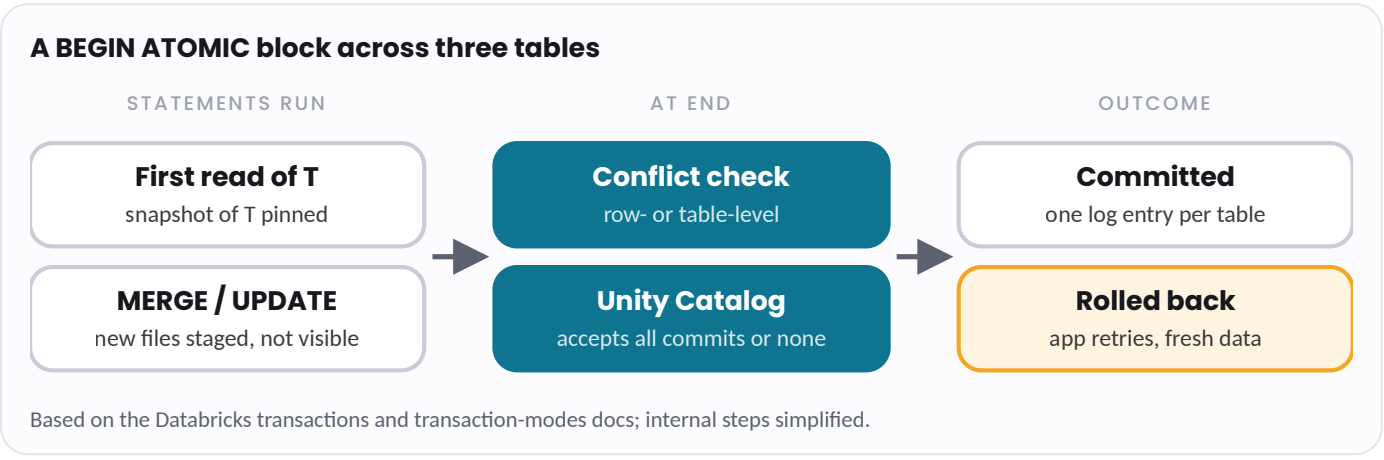
On Databricks you turn it on per table:[22]

```
CREATE TABLE sales_data (...)
TBLPROPERTIES ('delta.feature.catalogManaged' = 'supported');

-- or, on an existing managed table (DBR 18.0+):
ALTER TABLE sales_data
SET TBLPROPERTIES ('delta.feature.catalogManaged' = 'supported');
```

The docs warn that enabling it on a busy table "can take several minutes" while state is synced to the catalog, and not to cancel the `ALTER` halfway: an interrupted upgrade can leave the table locked until you rerun it.[22] Every table a transaction *writes* must be a Unity Catalog managed table with this feature.[20]

What happens inside a transaction



Reads are pinned per table. When a transaction first touches a table, "Databricks captures a consistent snapshot of that table at first access", and later reads of that table see the same snapshot even if others change it.[20] Note the "at first access": the snapshot is taken per table when you first read it, not once at `BEGIN`.

Writes are invisible until commit. In interactive mode, uncommitted changes are visible only to your session.[21] On success, each table gets **a single Delta log entry**, however many statements touched it.[20]

Conflicts are checked at the end. There are no locks. At commit time Databricks looks for write-write conflicts, write-read conflicts (under Serializable only), phantom reads and metadata conflicts. If you lose, "a failed transaction must be retried by the application with fresh data."[20]

How fine-grained the check is depends on the mode. This is the most important design detail:[21,20]

- `BEGIN ATOMIC` **(non-interactive)** uses **row-level concurrency** *≈ conflict detection that only fails two concurrent writes if they modify the same row, rather than the same data file* where the target table supports it. Two jobs updating different customers in the same table can both commit.
- `BEGIN TRANSACTION` **(interactive)** is more conservative: it "can conflict at the table level, except for unconditional appends."

Row-level concurrency itself has requirements: Databricks Runtime 14.3 LTS or later, an unpartitioned table, and **deletion vectors** *≈ a Delta feature that marks rows as deleted in a small side file instead of rewriting the whole Parquet file* enabled. When those hold, it's on automatically. The default **isolation level** *≈ how strictly concurrent transactions are kept from seeing or interfering with each other's changes* is **WriteSerializable**, with Serializable as the stricter option.[23] (Under WriteSerializable, an `INSERT` racing an `UPDATE` doesn't conflict; under Serializable it can.[23])

Where it runs

`BEGIN ATOMIC` runs on SQL warehouses, serverless compute, or clusters on **Databricks Runtime 18.0+**, from SQL, Scala or PySpark `spark.sql`. Interactive transactions run **only on SQL warehouses**, SQL only, and through drivers: JDBC 3.0.5+ with `setAutoCommit(false)`, ODBC 2.10.0+, or the Python connector with `autocommit=False`.[20,21] Disconnecting rolls back either kind.[20]

WHAT IT MEANS FOR YOU

If you're migrating from a warehouse, this removes one of the most common blockers. Stored procedures that assumed `BEGIN` / `COMMIT` semantics can move with fewer rewrites, which is clearly the market Databricks is after. But compare semantics, not just syntax. Snowflake's standard tables use READ COMMITTED and lock so that concurrent `UPDATE` / `DELETE` / `MERGE` on the same table generally block each other; DDL implicitly commits.[27] Databricks gives you per-table snapshots, no locks, and failure-and-retry at commit, and DDL isn't allowed inside a transaction at all.[20] Code that relied on waiting for a lock now needs a retry loop.

If you run multi-step ETL, wrap the steps that must agree, and keep them short. Every table and statement inside widens the window for a conflict at the end.

The staging-and-validation pattern the docs recommend is worth stealing, because it turns data-quality checks into a transaction boundary:

Write-audit-publish: before vs with transactions

Before: separate commits

```
INSERT INTO stage SELECT ...;
-- check runs; if bad, someone
-- must clean stage by hand
MERGE INTO dim_customer ...;
INSERT INTO fact_sales ...;
-- fact fails: dim already
-- committed, now inconsistent
UPDATE etl_meta SET ...;
```

After: one atomic block

```
BEGIN ATOMIC
  INSERT INTO stage SELECT ...;
  IF (SELECT count(*) FROM stage
      WHERE amount < 0) > 0 THEN
    SIGNAL SQLSTATE '45000';
  END IF;
  MERGE INTO dim_customer ...;
  INSERT INTO fact_sales ...;
  UPDATE etl_meta SET ...;
END;
```

Illustrative. SIGNAL SQLSTATE '45000' forcing a rollback is the documented pattern; check exact SQL scripting syntax for your runtime.

If you have external writers, catalog commits change the rules. Spark jobs outside Databricks that write straight to the table path will stop working: path-based access fails with `PATH_BASED_ACCESS` inside transactions, and catalog-managed tables don't support filesystem-only access at all.[20,25] External engines must go through Unity Catalog.

The hard limits, all from the docs:[20]

- at most **100 tables** read or written, plus up to **100 views** read, per transaction;
- at most **100 intermediate commits per table**;
- interactive transactions roll back after **10 minutes idle**; every transaction rolls back after **48 hours**;
- no DDL (`CREATE` , `ALTER` , `DROP TABLE`), no time travel, no `SHOW TABLES` or system-table queries inside;
- non-transactional sources (external Parquet, CSV, JDBC federation) can be read only with the `allow_nontransactional_read` hint, and those reads "are not repeatable".

💥 Watch out for

- **Iceberg status is contradictory.** The GA blog says transactions are GA for managed Delta *and* Iceberg tables.[18] The transactions docs and catalog commits page still say "Transactions that write to Unity Catalog managed Iceberg tables are in Private Preview."[20,22] Until the docs change, treat Iceberg writes as preview.
- **Interactive mode conflicts at table level.** Two analysts' `BEGIN TRANSACTION` sessions updating different rows of one table can still collide. Use `BEGIN ATOMIC` for row-level detection.[21]
- **Identity columns block concurrency.** "Tables with identity columns don't support concurrent transactions."[23]
- **Partitioned tables lose row-level concurrency.** It needs an unpartitioned table with deletion vectors.[23]
- **Streaming checkpoints aren't transactional.** You can run `BEGIN ATOMIC` inside `foreachBatch`, but "streaming checkpoints do not advance transactionally", so make the batch idempotent.[21]
- **Lineage events persist even if the transaction rolls back.**[20]
- **One transaction per connection.** Running transactions on several threads with one connection object is "undefined behavior".[21]
- **Turning it off isn't free.** Disabling catalog commits on an existing managed table needs DBR 18.0+, and open-source Delta says that once enabled, the feature "cannot be disabled".[22,25]

WHAT HAPPENS NEXT

Three things to watch. First, whether the docs catch up with the blog on Iceberg. Second, external engines: Databricks made external writes to catalog-managed Delta tables GA on 13 October,[28] but the catalog commits page still labels external access Beta,[22] so check before you rely on a non-Databricks writer. Third, the open-source side: the `catalogManaged` protocol is in Delta Lake and designed for "any catalog implementation" that supports it,[25] which leaves room for other catalogs to offer the same multi-table guarantees. If they do, the "first lakehouse" claim becomes a feature of the format rather than of one vendor.

📖 Words from this story

ABAC: attribute-based access control, permissions driven by tags on data and users, used here for row filters and column masks.

Lakehouse: an architecture that stores tables as open-format files (such as Parquet) in cheap object storage, with a metadata layer that adds database features on top.

Parquet: a column-oriented file format for analytics; each file stores data column by column, compressed.

Delta Lake: an open table format that turns a directory of Parquet files into a transactional table by keeping an ordered log of changes next to them.

Transaction log: Delta's `_delta_log` directory: numbered JSON files, one per commit, each listing files added and removed; the table's state is the sum of the log.

Optimistic concurrency control: a strategy where transactions run without locks and check for conflicts only when they try to commit; the loser retries or fails.

Catalog-managed tables: Delta tables whose commits are coordinated by a catalog service rather than by atomic file creation in storage.

Row-level concurrency: conflict detection that only fails two concurrent writes if they modify the same row, rather than the same data file.

Deletion vectors: a Delta feature that marks rows as deleted in a small side file instead of rewriting the whole Parquet file.

Isolation level: how strictly concurrent transactions are kept from seeing or interfering with each other's changes.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **psqlODBC 18.00.0005 (14 Oct):** fixes CVE-2026-109448 (CVSS 5.9), where binding more than 32,768 parameters wrote zeros outside the bindings array in versions 08.01.0102–18.00.0004; a long `IN (?, ?, ...)` list built from user input is the risky pattern.[29]
- **MongoDB 9.0.4 (13 Oct):** fixes five CVEs and several resharding deadlocks, but the `listIndexes` collation issue (SERVER-89953) that makes Mongoose `syncIndexes()` rebuild indexes is still open.[30]
- **pglifecycle 2.0 beta (14 Oct):** Gavin M. Roy's Rust rewrite of a tool that keeps a Postgres schema in Git as one YAML file per object and generates plan-first DDL; it has run AWeber's schema for over seven years.[31]
- **Databricks external writes to managed Delta tables GA (13 Oct):** external Spark clients can write to tables with catalog commits enabled.[32]
- **Databricks agentic code converter GA for SQL (15 Oct):** converts SQL from several dialects into Databricks SQL and validates the output.[32]
- **Kafka 4.4.0 still not released:** the downloads page's newest release by date is still 4.2.2 (29 Sep); highest version 4.3.1.[33]
- **DuckDB 2.0.0 still scheduled for 21 Oct (tentative),** with 2.0.1 on 18 Nov; latest release is 1.5.6 (28 Sep).[34]

1. **PgBouncer 1.26.1 released - Fixes three CVEs**, PgBouncer project on postgresql.org, 14 Oct 2026.
2. **PgBouncer home page news**, pgbouncer.org, viewed 18 Oct 2026.
3. **PgBouncer changelog**, entries for 1.26.1 "Only startup once" (14 Oct 2026) and 1.26.0 (23 Sep 2026).
4. **CVE-2026-107350**, OpenCVE (CNA: PostgreSQL), published 14 Oct 2026, viewed 18 Oct 2026.
5. **CVE-2026-107350**, ThreatInt CVE record (credits, reserved/published dates), viewed 18 Oct 2026.
6. **CVE-2026-107349**, OpenCVE, published 14 Oct 2026.
7. **CVE-2026-107348**, OpenCVE, published 14 Oct 2026.

8. **PgBouncer configuration**, pgbouncer.org documentation.
9. **pgbouncer authentication made easy**, Laurenz Albe, CYBERTEC, 2019 (updated Feb 2026).
10. **Nobody Patches the Pooler**, Christophe Pettus, The Build, 2 Oct 2026.
11. **Patch PgBouncer Today**, Christophe Pettus, The Build, 19 May 2026.
12. **Message Flow (Start-up)**, PostgreSQL 18 documentation, section 54.2.1.
13. **SASL Authentication**, PostgreSQL documentation.
14. **RFC 5802: Salted Challenge Response Authentication Mechanism (SCRAM)**, IETF, 2010, section 3.
15. **pgbouncer/pgbouncer releases** (1.25.2, 8 May 2026) and **pgbouncer.github.io changelog** (release dates of 1.17.0–1.25.2).
16. **pgbouncer, Debian Package Tracker**, viewed 18 Oct 2026.
17. **PgBouncer in Azure Database for PostgreSQL**, Microsoft docs, updated 22 Jul 2026.
18. **Transactions are Generally Available, advancing warehousing on the Lakehouse**, Benjamin Mathew, Prakhar Jain and Sumeet Varma, Databricks blog, 16 Oct 2026.
19. **Transactions on the open lakehouse unlock mission-critical warehousing workloads**, Databricks blog, 4 Mar 2026.
20. **Transactions**, Databricks documentation (AWS), viewed 18 Oct 2026.
21. **Transaction modes**, Databricks documentation (AWS), viewed 18 Oct 2026.
22. **Catalog commits**, Databricks documentation (AWS), viewed 18 Oct 2026.
23. **Row-level concurrency**, Databricks documentation (AWS).
24. **Diving Into Delta Lake: Unpacking The Transaction Log**, Burak Yavuz, Michael Armbrust and Brenner Heintz, Databricks blog, 21 Aug 2019.
25. **Use catalog-managed tables**, Delta Lake documentation.
26. **Multi-Table Transactions with Iceberg REST Catalogs and Apache Polaris**, Alex Merced, Dremio blog, 25 Sep 2026.
27. **Transactions**, Snowflake documentation.
28. **Databricks platform release notes, October 2026**, entry dated 13 Oct 2026.
29. **psqlODBC 18.00.0005 released with security fix for CVE-2026-109448**, postgresql.org, 14 Oct 2026.
30. **MongoDB 9.0 release notes: patch releases**, MongoDB docs, viewed 18 Oct 2026.
31. **Introducing pglifecycle 2.0 beta**, postgresql.org, 14 Oct 2026.
32. **Databricks platform release notes, October 2026**, entries dated 13 and 15 Oct 2026.
33. **Apache Kafka downloads**, viewed 18 Oct 2026.
34. **DuckDB release calendar**, viewed 18 Oct 2026.